ECM2433

Ollie Mock Paper 2

The C Family

Closed Book

Additional Materials: None

Permitted Materials: No calculators permitted

Guidelines:

The duration of this paper is 2hr 00.

Answer ALL questions.

Write all answers in the answer booklet provided.

Use only black pen or pencil.

Mobile and electronic devices are not allowed.

This converted mock paper has been checked so each question is answerable and answers are not given away in the question text.

Answers and marking notes are at the end of this document.

Question 1

(a) A programmer is writing a C function to manage a dynamic list of student IDs. (i) Identify the logic error in the loop and explain the likely runtime consequence. (ii) If compiled with AddressSanitizer, how would the report differ from a standard run? (4 marks)

```
#include <stdlib.h>
int *manage_list(int size) {
    int *list = malloc(size * sizeof(int));
    for (int i = 0; i <= size; i++) {
        list[i] = i * 10;
    }
    return list;
}
```

(b) Explain the difference between sizeof and strlen for the declaration below. (4 marks)

```
#include <string.h>
char hello[20] = "Hello world";
```

(c) In C++, how does an extern "C" declaration affect name mangling, and why is it needed when calling a function compiled from a C source file? (4 marks)

(d) Write a C function void swap_anything(void *p, void *q, size_t size) that swaps the contents of two memory locations of any data type. (8 marks)

(e) A programmer uses scanf("%s", buffer) to read a string. Explain why this is dangerous and suggest a better format string to prevent buffer overflow. (5 marks)

(Total 25 marks)

Question 2

(a) Consider the class hierarchy below. (i) If a programmer executes Sensor *s = new TempSensor("T1", 25.0); delete s;, explain why the output or cleanup may be incomplete and how to fix it. (ii) Explain static and dynamic binding in this context. (6 marks)

```
#include <iostream>
#include <string>
class Sensor {
public:
    std::string name;
    Sensor(std::string n) : name(n) {}
    virtual double read() { return 0.0; }
    ~Sensor() { std::cout << "Destructing Sensor"; }
};
class TempSensor : public Sensor {
    double reading;
```

```
public:
    TempSensor(std::string n, double r) : Sensor(n), reading(r) {}
    double read() override { return reading; }
    ~TempSensor() { std::cout << "Destructing TempSensor"; }
};
```

(b) C++ supports function overloading. Explain how name mangling lets the compiler support two functions with the same name but different parameter types. (4 marks)

(c) A developer uses std::cin >> age >> course to read 21 and "Computer Science". Explain why course only contains "Computer" and show the C++ function used to read the full line. (4 marks)

(d) Using the C++ Standard Template Library, write a snippet that takes a std::vector<Student> and sorts it by mark descending, then by name ascending for students with equal marks. Use a lambda expression. (6 marks)

```
struct Student {
    std::string name;
    int mark;
};
```

(e) Define a simple template class Pair<T> that holds two values of type T and provides a method bool are_equal() that compares the stored values using ==. (5 marks)

(Total 25 marks)

Question 3

(a) Option<T> and Result<T, E> handle missing values and errors. (i) Write a Rust function signature for find_task that takes a slice of Strings and a keyword, returning the index of the first match. (ii) Show how to handle the return value with match. (5 marks)

(b) Explain interior mutability in Rust, specifically referring to the relationship between Arc<T> and Mutex<T> in multi-threaded programs. (5 marks)

(c) Explain why the Rust code below fails to compile, referring to references, mutability, and possible Vec reallocation. (6 marks)

```
fn main() {
    let mut tasks = vec![String::from("Task 1")];
    let r1 = &tasks[0];
    tasks.push(String::from("Task 2"));
    println!("{}", r1);
}
```

(d) Write a Rust expression using iter, filter, map, and collect that takes a Vec<i32>, keeps only values greater than 10, doubles them, and returns a new Vec. (5 marks)

(e) What is deref coercion in Rust, and how does it allow a Box<Song> to be passed to a function expecting &Song? (4 marks)

(Total 25 marks)

Question 4

(a) Compare error handling in C, C++, and Rust. (i) Describe the disadvantage of C return codes when a valid integer could also be an error code. (ii) Explain how Rust's ? operator improves on manual checking of Result values. (6 marks)

(b) In C++, what are smart pointers? Contrast std::unique_ptr and std::shared_ptr, explaining how they relate to ownership. (6 marks)

(c) For a Rust parallel task, explain why thread::spawn often needs a move closure, and describe how an mpsc channel can collect results from multiple worker threads. (8 marks)

(d) In C++, give the prototype for overloading the + operator to add an integer number of seconds to a custom MyTime. (5 marks)

(Total 25 marks)

Answers and marking notes

Equivalent correct code and reasoning should receive credit.

Question 1

(a) Question (4 marks): A programmer is writing a C function to manage a dynamic list of student IDs. (i) Identify the logic error in the loop and explain the likely runtime consequence. (ii) If compiled with AddressSanitizer, how would the report differ from a standard run?

Code from question:

```
#include <stdlib.h>
int *manage_list(int size) {
    int *list = malloc(size * sizeof(int));
    for (int i = 0; i <= size; i++) {
        list[i] = i * 10;
    }
    return list;
}
```

Answer: The loop should use i < size, not i <= size. If size elements are allocated, valid indexes are 0 to size - 1, so list[size] writes one past the allocation. That is a heap buffer overflow and gives undefined behaviour. AddressSanitizer instruments the program and should stop with a clear heap-buffer-overflow report, often including the bad address and source location, instead of silently continuing or crashing unpredictably.

(b) Question (4 marks): Explain the difference between sizeof and strlen for the declaration below.

Code from question:

```
#include <string.h>
char hello[20] = "Hello world";
```

Answer: sizeof hello is 20 because it gives the storage size of the whole char array. strlen(hello) is 11 because it counts visible characters before the first null terminator. strlen does not include the terminator or unused remaining array capacity.

(c) Question (4 marks): In C++, how does an extern "C" declaration affect name mangling, and why is it needed when calling a function compiled from a C source file?

Answer: C++ normally mangles names so overloaded functions get unique linker symbols. C compilers do not use C++ name mangling. extern "C" tells the C++ compiler to use C linkage for the declaration, so the linker can match the C++ call with the symbol emitted by the C compiler.

```
extern "C" {
    void c_function(void);
}
```

(d) Question (8 marks): Write a C function void swap_anything(void *p, void *q, size_t size) that swaps the contents of two memory locations of any data type.

Answer: The function must treat the two objects as raw bytes because void pointers cannot be directly dereferenced. Copy p into temporary storage, q into p, then the temporary bytes into q. If malloc is used, check it and free it.

```
#include <stddef.h>
#include <stdlib.h>
#include <string.h>
void swap_anything(void *p, void *q, size_t size) {
    unsigned char *tmp = malloc(size);
    if (tmp == NULL) {
        return;
    }
    memcpy(tmp, p, size);
    memcpy(p, q, size);
    memcpy(q, tmp, size);
    free(tmp);
}
```

(e) Question (5 marks): A programmer uses scanf("%s", buffer) to read a string. Explain why this is dangerous and suggest a better format string to prevent buffer overflow.

Answer: %s reads until whitespace with no automatic knowledge of the buffer size. If the input word is too long, it writes past the array. Use a width limit that leaves room for the null terminator, for example scanf("%19s", buffer) for char buffer[20]. fgets(buffer, sizeof buffer, stdin) is often better for reading a whole line.

Question 2

(a) Question (6 marks): Consider the class hierarchy below. (i) If a programmer executes Sensor *s = new TempSensor("T1", 25.0); delete s;, explain why the output or cleanup may be incomplete and how to fix it. (ii) Explain static and dynamic binding in this context.

Code from question:

```
#include <iostream>
#include <string>
```

```
class Sensor {
public:
    std::string name;
    Sensor(std::string n) : name(n) {}
    virtual double read() { return 0.0; }
    ~Sensor() { std::cout << "Destructing Sensor"; }
};
class TempSensor : public Sensor {
    double reading;
public:
    TempSensor(std::string n, double r) : Sensor(n), reading(r) {}
    double read() override { return reading; }
    ~TempSensor() { std::cout << "Destructing TempSensor"; }
};
```

Answer: Deleting through a base pointer requires a virtual base destructor. Without it, deleting Sensor * that points to a

```
TempSensor has undefined behaviour and may not run the derived destructor. Fix it with virtual ~Sensor() = default; or a
```

Answer: virtual destructor body. read is virtual, so s->read() uses dynamic binding and calls TempSensor::read() at runtime. Static binding is compile-time selection from the static type; dynamic binding is runtime virtual dispatch based on the real object type.

(b) Question (4 marks): C++ supports function overloading. Explain how name mangling lets the compiler support two functions with the same name but different parameter types.

Answer: Overloaded functions have the same source name but different parameter lists. The linker still needs distinct symbols, so the C++ compiler encodes information such as the function name and parameter types into the generated symbol names. The exact mangled names are compiler-specific.

(c) Question (4 marks): A developer uses std::cin >> age >> course to read 21 and "Computer Science". Explain why course only contains "Computer" and show the C++ function used to read the full line.

Answer: operator>> for std::string reads one whitespace-separated token, so it stops before Science. Use

```
std::getline(std::cin, course) to read a full line. After reading age with >>, discard the remaining newline before calling
```

Answer: getline.

```
int age;
std::string course;
std::cin >> age;
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
std::getline(std::cin, course);
```

(d) Question (6 marks): Using the C++ Standard Template Library, write a snippet that takes a std::vector<Student> and sorts it by mark descending, then by name ascending for students with equal marks. Use a lambda expression.

Code from question:

```
struct Student {
    std::string name;
    int mark;
};
```

Answer: The comparator should return true when the first student should appear before the second. Higher marks come first; equal marks use name alphabetical order.

```
std::sort(students.begin(), students.end(),
    [](const Student& a, const Student& b) {
        if (a.mark != b.mark) {
            return a.mark > b.mark;
        }
        return a.name < b.name;
    });
```

(e) Question (5 marks): Define a simple template class Pair<T> that holds two values of type T and provides a method bool are_equal() that compares the stored values using ==.

Answer: Pair stores two T values. are_equal is a normal member function; it relies on operator== being valid for T.

```
template<typename T>
class Pair {
    T first;
    T second;
public:
    Pair(T first, T second) : first(first), second(second) {}
    bool are_equal() const {
        return first == second;
    }
};
```

Question 3

(a) Question (5 marks): Option<T> and Result<T, E> handle missing values and errors. (i) Write a Rust function signature for find_task that takes a slice of Strings and a keyword, returning the index of the first match. (ii) Show how to handle the return value with match.

Answer: The missing case should be represented by Option<usize>. The match must handle both Some(index) and None.

```
fn find_task(tasks: &[String], keyword: &str) -> Option<usize> {
    tasks.iter().position(|task| task.contains(keyword))
}
```

Answer: match find_task(&tasks, "urgent") { Some(index) => println!("found at {index}"), None => println!("not found"),

```
}
```

(b) Question (5 marks): Explain interior mutability in Rust, specifically referring to the relationship between Arc<T> and Mutex<T> in multi-threaded programs.

Answer: Interior mutability means a wrapper lets you mutate inner data through a shared outer value while enforcing rules at runtime. Arc gives thread-safe shared ownership using atomic reference counting. Mutex protects the value so only one thread at a time can access it mutably. Arc<Mutex<T>> is the common combination for shared mutable state across threads.

(c) Question (6 marks): Explain why the Rust code below fails to compile, referring to references, mutability, and possible Vec reallocation.

Code from question:

```
fn main() {
    let mut tasks = vec![String::from("Task 1")];
    let r1 = &tasks[0];
    tasks.push(String::from("Task 2"));
    println!("{}", r1);
}
```

Answer: r1 is an immutable reference into tasks. push requires mutable access to tasks. Rust does not allow a mutable borrow while an immutable borrow that is later used is still active. Also, pushing to a Vec may reallocate its buffer, which could invalidate references to old elements. Use r1 before pushing, or clone the string if it must be kept independently.

(d) Question (5 marks): Write a Rust expression using iter, filter, map, and collect that takes a Vec<i32>, keeps only values greater than 10, doubles them, and returns a new Vec.

Answer: With iter, the filter closure receives a reference to an item reference, so the filter uses **n. The map closure receives the iterator item reference, so *n gives the integer.

```
let result: Vec<i32> = values
    .iter()
    .filter(|n| **n > 10)
    .map(|n| *n * 2)
    .collect();
```

(e) Question (4 marks): What is deref coercion in Rust, and how does it allow a Box<Song> to be passed to a function expecting &Song?

Answer: Box<T> owns a T on the heap and implements Deref<Target = T>. A borrowed Box<Song> can therefore be coerced from &Box<Song> to &Song when a function expects &Song. This borrows the inner Song; it does not move ownership out of the Box.

Question 4

(a) Question (6 marks): Compare error handling in C, C++, and Rust. (i) Describe the disadvantage of C return codes when a valid integer could also be an error code. (ii) Explain how Rust's ? operator improves on manual checking of Result values.

Answer: C return codes can be ambiguous: a value such as 0 or -1 may be both a valid result and an error signal unless extra state is provided. C++ can use exceptions for exceptional failure. Rust uses Result<T, E> for recoverable errors, so success and failure are explicit. The? operator unwraps Ok values and returns Err early, avoiding repeated manual match code while preserving the error.

(b) Question (6 marks): In C++, what are smart pointers? Contrast std::unique_ptr and std::shared_ptr, explaining how they relate to ownership.

Answer: Smart pointers are RAII objects that manage dynamically allocated objects. unique_ptr has exclusive ownership, cannot be copied, and destroys the object when it goes out of scope unless moved. shared_ptr has shared ownership with a reference count; the object is destroyed when the last shared_ptr owner is gone.

(c) Question (8 marks): For a Rust parallel task, explain why thread::spawn often needs a move closure, and describe how an mpsc channel can collect results from multiple worker threads.

Answer: A spawned thread may outlive the stack frame that created it, so values used by the thread often need to be moved into the closure. mpsc means multiple producer, single consumer. Clone tx for each worker, move that clone into the closure, send the worker result with tx.send(...).unwrap(), and receive results through rx in the main thread. Drop the original tx when no more sends will happen so receiver iteration can finish.

(d) Question (5 marks): In C++, give the prototype for overloading the + operator to add an integer number of seconds to a custom MyTime.

Answer: As a member operator, the left-hand operand is *this, so the operator takes only the integer seconds. The trailing const means adding seconds does not mutate the original object.

```
class MyTime {
public:
    MyTime operator+(int seconds) const;
};
```